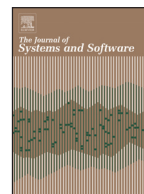




Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Test coverage of impacted code elements for detecting refactoring faults: An exploratory study

Everton L.G. Alves*, Tiago Massoni, Patrícia Duarte de Lima Machado

Federal University of Campina Grande, Rua Aprigio Veloso, 882, Bairro Universitário, Campina Grande, PB, Brazil

ARTICLE INFO

Article history:

Received 30 November 2014

Revised 5 January 2016

Accepted 1 February 2016

Available online xxx

Keywords:

Testing
Refactoring
Coverage

ABSTRACT

Refactoring validation by testing is critical for quality in agile development. However, this activity may be misleading when a test suite is insufficiently robust for revealing faults. Particularly, refactoring faults can be tricky and difficult to detect. Coverage analysis is a standard practice to evaluate fault detection capability of test suites. However, there is usually a low correlation between coverage and fault detection. In this paper, we present an exploratory study on the use of coverage data of mostly impacted code elements to identify shortcomings in a test suite. We consider three real open source projects and their original test suites. The results show that a test suite not directly calling the refactored method and/or its callers increases the chance of missing the fault. Additional analysis of branch coverage on test cases shows that there are higher chances of detecting a refactoring fault when branch coverage is high. These results give evidence that a combination of impact analysis with branch coverage could be highly effective in detecting faults introduced by refactoring edits. Furthermore, we propose a statistic model that evidences the correlation of coverage over certain code elements and the suite's capability of revealing refactoring faults.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

Refactoring improves quality factors of a program while preserving its external behavior (Fowler et al., 1999; Mens and Tourwé, 2004). Refactoring edits are one of the foundations of agile software development. In the agile community, the refactoring activity is known to confine the complexity of a source code, improving non-functional aspects of a software such as decreased coupling and increased cohesion (Moser et al., 2008). Fowler et al. (1999) lists four advantages that refactoring brings in the context of Agile Methods: (i) it helps developers to program faster; (ii) it improves the design of the software; (iii) it makes software easier to understand; and (iv) it helps developers to find bugs.

Recent studies have evidenced that nearly 30% of the changes performed during software development are likely to be refactorings (Soares et al., 2011). For example, code clones spread throughout several methods of a class can be unified into a single method, then replacing the clones by a call to this new method; this is the Extract Method refactoring (Fowler et al., 1999), which is one of the most widely applied (Murphy et al., 2006).

Although there are several automatic refactoring tools in popular IDEs, developers still perform most refactorings manually. Murphy et al. (2006) find that about 90% of refactoring edits are manually applied. Negara et al. (2013) agree by showing that expert developers prefer manual refactorings over automated. Usability issues seem to have a negative impact on developers' confidence on those tools (Lee et al., 2013). Moreover, recent works show that incorrect refactorings – unexpectedly changing behavior – are present even in the most used tools (Daniel et al., 2007; Soares et al., 2013).

In such scenario, developers widely use *regression test suites* for validating manually-applied refactoring edits. As such, refactoring edits are error prone and require validation, as subtle faults may pass unnoticed. Dig and Johnson (2005) state that nearly 80% of the changes that break client applications are API-level refactoring edits. In addition, 77% of the participants from Kim et al.'s survey with Microsoft developers (Kim et al., 2012) confirm that refactoring may induce the introduction of subtle bugs and functionality regression. A regression test suite, however, may be ineffective in finding refactoring faults. Also, it may be impractical to rerun and analyze the execution results of the whole test suite after each refactoring edit. Techniques that minimize the test suite, while maintaining its effectiveness, are desirable.

Nevertheless, this intuition has little scientific evidence; it is important to distinguish which impacted methods, if called by the

* Corresponding author. Tel.: +55 83 3310 1122.

E-mail addresses: everton@copin.ufcg.edu.br (E.L.G. Alves), tiago@computacao.ufcg.edu.br (T. Massoni), patricia@computacao.ufcg.edu.br (P.D.L. Machado).

test suite, are most effective in detecting faults that might be led to by refactoring. In this paper, we present an exploratory study, performed on three real open-source Java projects, with seeded faults related to two of the most common refactoring edits, Extract Method and Move Method. Using the actual test suites from the selected projects, we measure the *direct calls* (first-level coverage) to several groups of methods possibly impacted by a refactoring edit, relating these data to the status of the test case – whether it detects or not the seeded fault.

Overall, only 67% of the seeded 270 faults were detected by the project's test suite. The lack of test cases calling the method whose body is changed seem to be very relevant – 70% of the unrevealed faults present this property. Similarly, 51% of the undetected faults are missed by test cases that directly call the callers of the changed method. On the other hand, for 78% of the detected faults, the test suite included at least one test case that calls the refactored method directly. Considering callers, this rate was also high (70%). In 62% of these suites there were test cases that cover, at first level, both the refactored method and its callers. The detection results did not present statistical dependence with the type of refactoring (same with the type of seeded fault). Based on our results, we propose a statistical model that uses first-level coverage data to foresee chances a test suite has to detect refactoring faults.

First-level coverage reports on a direct need for the agile developer – identify the calls that must be made in a test case for improving its chance of detecting refactoring faults. On the other hand, indirect coverage of impacted elements may not be applicable in a agile context. The reason is that it can be tricky and demand high costs for a developer to assess fault detection capability in indirect calls. For instance, after applying an Extract Method, it seems intuitive that tests directly calling the changed method, its callers, and callees present good chances of detecting any newly-introduced fault. When considering first-level coverage, we are also focusing on test case expressiveness regarding refactoring edits. When a test case that calls directly a method fails, it may be more helpful to locate the fault.

Considering that several faults were missed by test cases even with first-level coverage of impacted methods, we additionally analyzed test cases that exercise the modified method and their callers. If at least one test case in the suite called the changed method, and the branch coverage of this method was greater than 75%, 91% of the faults were detected. If callers of the changed method were directly accessed, in 88% of the cases, the faults were detected with high branch coverage. For suites with low branch coverage (less than 25%), detection dropped to 66% and 62%, respectively. These results provide a good case for tests with direct calls combined with high branch coverage.

As another additional study, we explored the relationship between first-level coverage of impacted elements and binding issues with refactored variables within class hierarchies. Previous research (Soares et al., 2011) reported on several subtle faults in manual and automated refactoring being due to homonymous variables or methods being confused by refactored statements, so we extended our investigation for relating test cases with this kind of refactoring fault. Similarly to the other studies, this investigation showed that when a test suite covers the refactored method and its callers better are the chances of detecting binding-related faults introduced when refactoring.

We published a preliminary version of this study (Alves et al., 2014a) in which a single refactoring type and refactoring fault are analyzed. The current paper extends our previous study by investigating new refactoring types, new refactoring faults, by adding statistical validation to the conclusions, and by proposing new artifacts to help the evaluation of a test suite regarding its detection of refactoring faults.

Section 2 brings a motivating example for the problem of test cases that miss refactoring bugs. Next, we present the setup and research questions investigated by the experimental studies (Section 3), then Section 4 includes the results and discussion for the main experimental study. In Section 5, we extend the study to relate its results with branch coverage within the exercised methods, while Section 6 presents an exploratory study for binding-related refactoring faults. Section 7 discusses threats to validity. The last two sections cover the related work and concluding remarks, respectively.

2. Motivating example

In agile methodologies, even simple solutions may need improvement when requirement changes must be incorporated into the code base, so manual refactoring is frequent. Automation of refactoring is common, but here we focus on manual refactoring.

Opportunities for code improvement often involve code duplication, and its minimization or elimination is often desirable. For this task, the Extract Method refactoring (Fowler et al., 1999) encompasses small changes that group together multiple code fragments into a new method; the new method has a name explaining its purpose, and the original fragments are then replaced by a call to this method. When applying this edit, developers must be cautious: the new method must receive parameters that correspond to the variables manipulated by the grouped fragment, and a return value must be correctly provided to the callers; also, the new method could be changing the behavior of the target class.

Suppose that, after working on several tasks, a developer notices an opportunity of reducing code duplication. Fig. 1 presents two fragments of her code before and after the Extract Method refactoring; Lines 5–8 from `Elementm(boolean)` – Fig. 1(a) – are extracted into the `n` method in Fig. 1(b). Following principles for validating refactorings, and assuming that her test suite passes before and after the edits, also, no compilation error was found. Thus, she may become confident that the modification is correct, and commits the code to the code base. The *behavior*, however, is undesirably modified. If `b` is true, `x` finishes with value 23, `x` is updated before throwing the exception. After extracting the method, the exception is thrown with the global `x` with its initial state 42.

This example shows a very subtle refactoring fault that may easily go undetected, especially if the test suite does not exercise the test method. By examining previous research on change impact analysis (Ren et al., 2004; Zhang et al., 2011; Ryder and Tip, 2001), we can establish that, for an Extract Method refactoring edit, the methods most likely to be impacted are: (i) *the original method (M)*, in this case, the `m` method; (ii) *the callers of the method under refactoring (C)*, as methods that call `m()` might be negatively influenced in case they use `m`'s return value, and/or any variable handled by `m`; (iii) *the callees of the method under refactoring (Ce)*, so if given methods that `m` calls require as pre-requisite the program to be in a certain state, then `m` must be run according to its previous behavior; and (iv) *methods with similar signature to the newly added one (O)*. An extracted method may break or introduce overriding/overloading contracts causing a behavior change; for instance, `m` could already be declared within `Element`'s hierarchy.

It is expected that, by presenting first-level coverage of the methods potentially impacted by the refactoring, a test case is expected to detect such fault – this is an easy to follow guideline for writing appropriate test cases. Nevertheless, first-level coverage alone may mislead developers/testers (Inozemtseva and Holmes, 2014). More specifically, with refactoring faults, there is no evidence whether this type of coverage is a good quality measure in this context, or which methods a test suite should call to

```

1 class Element{
2   int m(boolean b){
3     int x = 42;
4     try{
5       if (b){
6         x = 23;
7         throw new Exception();
8       }
9     }catch (Ex e){
10      return x;
11    }
12    return x;
13  }
14  int test(){
15    return m(true);
16  }
17  ...
18 }

(a) Original version.

1 class Element{
2   int m(boolean b){
3     int x = 42;
4     try{
5       x = n(b, x);
6     }catch (Ex e){
7       return x;
8     }
9     return x;
10  }
11  int n(boolean b, int x) throws
      Exception{
12    if (b){
13      x = 23;
14      throw new Exception();
15    }
16    return x; }
17  int test(){
18    return m(true);
19  }
20  ...
21 }

(b) Code after a problematic Extract Method
    edit.

```

Fig. 1. Extract Method Example. Lines 6–9 are extracted to *n*.

consider it acceptable for validating an edit. One may argue that, for method-level refactorings, tests that call the refactored method directly are sufficient for detecting potential refactoring faults, but this assumption is questionable. Considering code from Fig. 2a, a developer extracts lines 8–11 to a new method `newMethod` (Fig. 2c), although, simultaneous to the refactoring, she sees a way to make the logic expression (`threshold > 10`) easier to understand. Then, she seizes the chance of, while moving/extracting a method, also improving the conditional statement. In this example, she exchanges the logical operator, however, the bodies within the conditional are not inverted accordingly (line 13– Fig. 2b), resulting in a different (and incorrect) behavior.

Two test cases, depicted in Fig. 2c (`testCreateItem` and `testCalculateNewValue`), can be written in order to detect potential refactoring faults. Test case `testCreateItem` calls the refactored method `createItem`, but misses the fault, even though all `createItems` branches are covered by the two calls in the test case, due to the test data. This fault is revealed only by `testCalculateNewValue`, in which the caller of `createItem`, `calculateNewValue`, is directly called. As the change also impacted (`calculateNewValue`), the test case that called it ends up detecting the problem as a *side effect*. A test suite with tests presenting first-level coverage of the refactored method might give to the developer a false sense of proper refactoring validation, where in fact it might not detect certain faults. Thus, for refactoring validation, there is a need of having test cases that exercise different elements of the code, not only the ones directly involved with the refactoring.

Our study aims at collecting and analyzing detailed evidence regarding the relationship between direct test calls to several categories of methods and their effectiveness in detecting refactoring-related seeded faults, within a controlled environment. As relevant examples of refactoring edits, we observed both Extract Method and Move Method (Fowler et al., 1999), being applied to open-source Java projects and their real JUnit test suites.

3. Experimental setup

3.1. Research questions

We conducted three exploratory studies with real open source programs to address the following research questions. We discuss RQ1–RQ4 in the first study (Section 4), while we discuss RQ5 and RQ6 in Sections 5 and 6, respectively.

1. RQ1: Is the type of refactoring a factor that influence a suite's capacity of detecting refactoring faults?
2. RQ2: Is the type of refactoring fault a factor that influence a suite's capacity of detecting refactoring faults?
3. RQ3: Are direct calls to the most commonly impacted elements appropriate to evaluate a test suite's capability of detecting refactoring faults?
4. RQ4: Which impacted elements are, when called by the test case, most likely to reveal refactoring faults?
5. RQ5: What is the additional influence of branch coverage on the detection of refactoring faults by test cases?
6. RQ6: What is the answer to RQ3 if the faults involve dynamic binding changes in subtypes?

We selected three open source Java projects as experimental units: XML-Security¹ (≈ 17 KLOC), a library that provides security APIs for manipulating XML documents, such as authorization and encryption; JMock² (≈ 5 KLOC), a library for testing applications with mock objects; and EasyAccept³ (≈ 2 KLOC), a tool that helps create and run acceptance tests. Those three systems present a common property: they include an active, up-to-date test suite, which is key to our study, as we intended to apply the real test base for each project, emulating its actual testing process. The

¹ <http://xml.apache.org/security>

² <http://jmock.org/>

³ <http://easyaccept.sourceforge.net/>

```

1 class Item{
2     int preSaleVal, appreciation;
3     public Item(int val1, int val2){
4         this.preSaleVal = val1;
5         this.appreciation = val2;
6     }
7     class Manager{
8         Item createItem(int val, int
9             threshold){
10             if (threshold > 10)
11                 return new Item(val,10);
12             else
13                 return new Item(val,threshold);
14         }
15     }
16     int calculateNewValue (int x,int y)
17     {
18         Item item = createItem(x, y);
19         return Math.pow(item.preSaleVal,
20             item.appreciation);
21     }
22 }

```

(a) Original version.

```

1 class Item{
2     int preSaleVal, appreciation;
3     public Item(int val1, int val2){
4         this.preSaleVal = val1;
5         this.appreciation = val2;
6     }
7     class Manager{
8         Item createItem(int val,
9             int threshold){
10             return newMethod(val,threshold);
11         }
12         Item newMethod(int val,
13             int threshold){
14             if (threshold < 10)
15                 return new Item(val,10);
16             else
17                 return new Item(val,threshold);
18         }
19     }
20     int calculateNewValue(int x, int y){
21         Item item = createItem(x, y);
22         return Math.pow(item.preSaleVal,
23             item.appreciation);
24     }
25 }

```

(b) Edited code.

```

1 void testCreateItem(){
2     Manager manager = new Manager();
3     Item item1 = manager.createItem(5,20);
4     Item item2 = manager.createItem(5,5);
5     assertEquals (item1.threshold,10);
6     assertEquals (item2.threshold,5);
7 }
8 void testCalculateNewValue(){
9     Manager manager = new Manager();
10    assertEquals (manager.calculateNewValue(5,20), 9765625);
11    assertEquals (manager.calculateNewValue(5,5), 3125);
12 }

```

(c) Test suite.

Fig. 2. Extract Method Example. Lines 6–8 are extracted to calculateNewValue.

largest test suites are in JMock – the number of test cases in each suite vary according to the collected version. For instance, Version 1 in JMock is supported by 195 test cases, while Version 2 presents 222 tests. Regarding versions, we randomly fetched three versions of each program from its repository history, with the respective test suites. Those versions were to be seeded with refactoring faults.

3.2. Refactorings and seeded faults

Our study focuses on two refactoring edits, Extract Method (EM) and Move Method (MM) (Fowler et al., 1999), two of the most applied refactorings in Java systems (Murphy et al., 2006); several IDEs provide their automatic application. They represent refactoring edits that involve a single class (in Extract Method, part of the body is cut and pasted into a new local method) or a pair of classes (in Move Method, a method is transferred to another class). Despite their apparent simplicity, potential issues must be addressed: state preservation of variables, fields and parameters being read or written by the statements extracted/moved, the value returned by the Extract Method, or breaking/adding

overriding/overloading constraints. Therefore, often faults are introduced when the steps are performed manually (Weißgerber and Diehl, 2006; Kim et al., 2011); nevertheless, neither refactoring tools are free from faults (Soares et al., 2010).

Due to the difficulty of mining refactoring faults that can be detectable by available test suites, we decided to use only seeded faults in our study. To represent erroneous refactoring edits, we seeded three types of faults: *missing statement* (MS), *relational operator replacement* (ROR), and *extra behavior* (EB). Several examples of these types of faults can be found in the experimental data used by work of Bavota et al. (2012). In their work, in three Java software systems (Ant, ArgoUML and Xerces) and their repositories, they identified 15,008 refactoring operations and related them to projects' bug fixes, in order to detect the refactorings that induced real bugs. From their study, we collected all reported bug-inducing refactorings edits. After selecting only the Extract Method and Move Method problematic refactorings, we downloaded the projects versions before and after each refactoring in order to visualize and understand bug causes. Most of the problematic Extract and Move Method edits can be classified among the three refactoring types (MS, ROR, and EB) considered in our study.

```

1 class Manifest{
2     void generateDigestValues() throws...{
3         if (this.state == MODESIGN) {
4             for (int i = 0;i < this.references.size();i++){
5                 Reference currentRef = (Reference) this.references.elementAt(i);
6                 currentRef.generateDigestValue();
7                 currentRef.setUnavailable();
8                 currentRef.restoreOriginalElements();
9             }
10            ...
11        }
12    }
13    ...
14 }

```

(a) original code

```

1 class Manifest{
2     void generateDigestValues() throws...{
3         if (this.state == MODESIGN) {
4             for (int i = 0;i < this.references.size();i++){
5                 Reference currentRef = (Reference) this.references.elementAt(i);
6                 applyModifications(currentRef);
7             }
8             ...
9         }
10    }
11    void applyModifications (Reference currentRef) throws...{
12        currentRef.generateDigestValue();
13        currentRef.setUnavailable();
14    }
15    ...
16 }

```

(b) extract method refactoring with missing statement

Fig. 3. An example of a Missing Statement fault.

The missing statement fault. In practice, developers perform Extract Method and Move Method by cutting statements from the source method, and pasting them to the target method. During this process, a developer might forget to paste one or more statements, changing the expected behavior. Similarly, the second step of Fowler's Extracted Method mechanics (Fowler et al., 1999) states that, in order to be systematic, a developer must "Copy the extracted code from the source method into the new target method". When performing this step, statement(s) might not be correctly transferred to the new method; this fault represents this situation. Fig. 3, from XML-Security, illustrates such fault. Line 6–8 in generateDigestValues() should have been extracted to applyModifications(Reference). However, while extracting the statements, a developer missed the last "paste" operation, forgetting to add statement currentRef.restoreOriginalElements(); to the newly created method (Fig. 3b). Although no compilation error is found, the system's previous behavior is modified.

Similarly, we also consider scenarios in which a developer, aiming to work systematically, tries to follow one of Fowler's principles (Fowler et al., 1999). Fowler states the need of breaking a refactoring transformation into a sequence of small steps, in order to decrease the chances of introducing faults. In this context,

a developer could, for instance, comment part of the newly added method's code and uncomment it, statement by statement, while searching for missing variables or references problems. For example, variables used only within this extracted code need to be declared as temporary. During this process, a developer might forget to uncomment a statement after he is done.

The relational operator replacement fault. Manual refactoring and other restructuring changes are often combined into a single commit (Murphy-Hill et al., 2012; Alves et al., 2014b). This fault represents the joint application of an Extract/Move Method with changes in a conditional statement configuration. For instance, we simulate a change in the order of an IF/ELSE command that, in order to preserve the behavior, requires the inversion of the boolean condition. As an abstraction of this case, our fault simply reverses the boolean condition of an IF command, altering the behavior of the method. This fault is exemplified in Fig. 4 in a Move Method refactoring on XML-Security. Method getProviderIsRegAtSec is moved from JCEMapper to Algorithm, along with the inversion of the IF/ELSE command (Fig. 4a, Lines 4–6). Fig. 4b shows what would be the correct resulting code. The scenario can be reproduced by only changing the relational operator (! = for ==) in

```

1  class JCEMapper{
2      boolean getProviderIsRegAtSec(String providerId) {
3          Provider prov = Security.getProvider(providerId);
4          if (prov != null) {
5              return true;
6          } else return false;
7      }
8  }
9  class Algorithm{...}

```

(a) original code

```

1  class JCEMapper{...}
2  class Algorithm{
3      boolean getProviderIsRegAtSec(String providerId) {
4          Provider prov = Security.getProvider(providerId);
5          if (prov == null) {
6              return false;
7          } else return true;
8      }
9  }

```

(b) Correct Move Method with inverted IF/ELSE.

```

1  class JCEMapper{...}
2  class Algorithm{
3      boolean getProviderIsRegAtSec(String providerId) {
4          Provider prov = Security.getProvider(providerId);
5          if (prov == null) {
6              return true;
7          } else return false;
8      }
9  }

```

(c) Incorrect Move Method with inverted IF/ELSE.

Fig. 4. Example of a Relational Operator Replacement fault.

Line 5, modifying the behavior without introducing compilation errors (Fig. 4c).

The extra behavior fault. Recent studies Kim et al. (2012); Murphy-Hill et al. (2012) show that developers often interleave refactorings with extra semantic changes. Hardly a commit contains a single refactoring edit; often, developers group into a single commit refactoring and non-refactoring edits. Non-refactoring edits (extra edits) usually change behavior, but they do not necessarily lead to refactoring faults – semantic changes may be intentional. Nevertheless, separating refactoring from extra edits could be beneficial. For instance, according to studies at Microsoft (Kim et al., 2012), developers report that they would like to see semantic changes that deviate from pure refactoring separately, especially when reviewing the correctness of refactorings. Fig. 5 illustrates such fault. Lines 4–8 are extracted to `iterateByCause`. However, besides the refactoring, an extra edit is performed, variable `strWriter` is

set to its default value (Fig. 5b - line 2), which interferes with the previous behavior, since this variable is used in the extracted code snippet.

For each system, each of the three randomly selected versions was subject to five faulty Extract Method edits each with a single Missing Statement, five with a Relational Operator Replacement, and five with Extra Behavior. The same process was repeated with Move Method. In total, we created 270 faulty versions; 90 for each system, 45 with Extract Method and 45 with Move Method. Fig. 6 presents the configuration of our study for a single system (experimental unit).

The chosen number of faults (270) is due to the difficulty of automatically mutating the code without generating refactored programs that are uncompileable. We aim to study a suite's capacity of revealing refactoring faults that might pass unnoticed, thus our fault injecting processes discard any output code that includes compilation errors. Therefore, the number of possible locations to inject the faults are limited.

```

1 String exceptionToString(Throwable exception) {
2     StringWriter strWriter = new StringWriter();
3     PrintWriter printWriter = new PrintWriter(strWriter);
4     do {
5         printWriter.println();
6         exception.printStackTrace(printWriter);
7         exception = exception.getCause();
8     }while(exception!= null);
9     printWriter.flush();
10    strWriter.flush();
11    return strWriter.toString();
12 }

```

(a) original code

```

1 String exceptionToString(Throwable exception) {
2     StringWriter strWriter = null;
3     PrintWriter printWriter = new PrintWriter(strWriter);
4     exception = iterateByCause(exception, printWriter);
5     printWriter.flush();
6     strWriter.flush();
7     return strWriter.toString();
8 }
9 Throwable iterateByCause(Throwable exception, PrintWriter printWrite){
10    do {
11        printWriter.println();
12        exception.printStackTrace(printWriter);
13        exception = exception.getCause();
14    }while(exception!= null);
15    return exception;
16 }

```

(c) Extract Method with an extra behavior.

Fig. 5. Example of an Extra Behavior fault.

Each of the 270 faulty versions differs from the original only by a single erroneous refactoring edit, similar to the examples in Figs. 3–5. As our study deals with two refactoring types and two refactoring faults, we establish a fault seeding process for each combination of *refactoring X fault* (Tables 1 and 2). Each injects a refactoring related mutant that is compilable. Although dealing only with injected faults, we believe the results from our study can relate to real world faults. Recent studies corroborate to this expectation: Just et al. (2014) find a statistically significant correlation between mutant detection and real fault detection, independently of code coverage; Andrews et al. (2005) find that mutation operators yield trustworthy results for testing experiments. Moreover, seeded faults have been widely used in testing related empirical studies (e.g., Kim et al., 2001; Memon et al., 2003; Andrews and Zhang, 2003; Briand et al., 2004).

3.3. Metrics

We ran each project's test suite on its three versions, for all 270 faulty versions. Then, we measured the following coverage properties: M : number of test cases that directly invoke the refactored method; C : number of test cases that directly invoke the callers of the refactored method; C_e : number of test cases that directly invoke the callees of the refactored method; and O : number of test cases that directly invoke methods with similar signature of

the newly added methods (by similar signature we mean methods with the same name and return type). We also measure how many of the test cases in fact detect the refactoring fault, for each of the defined groups: D_M : number of test cases, counted in M , that detect the fault; D_C , D_{C_e} and D_O are defined accordingly.

For calculating branch coverage, we use the Eclemma tool.⁴ Details about artifacts used in our study (subjects, faults and metrics) are available at our website.⁵

4. Study on test coverage for impacted program elements

The discussion in this section considers research questions RQ1–4, analyzing the influence of refactoring or fault type on the suite effectiveness, and the statistical model that approximates the chance of detecting a fault by means of the first-level coverage of test cases.

4.1. Refactoring type and fault influence on test effectiveness

All versions were subject to fault seeding; they all present at least 24% of undetected faults, as summarized in Table 3. In fact,

⁴ <http://www.eclemma.org/>

⁵ <https://sites.google.com/site/coverageexperiment/>

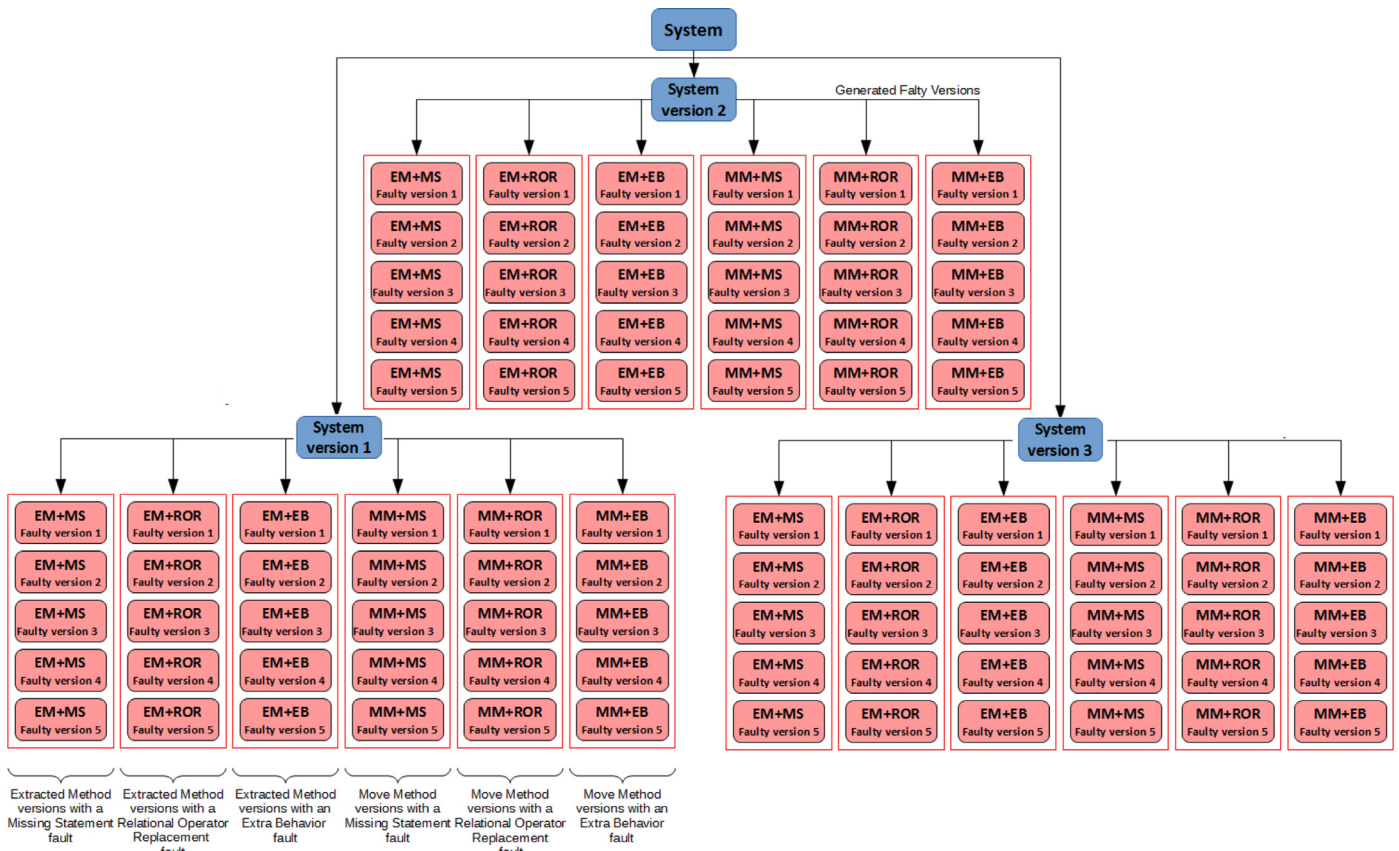


Fig. 6. Study configuration for a single program.

Table 1
Fault seeding on Extract Method.

Seeding a <i>a missing statement</i> fault in an Extract Method edit: Being <i>S</i> the set of classes from a given project version.
1. Randomly select a class <i>C</i> from <i>S</i> ;
2. Randomly select a method <i>m</i> from <i>C</i> ; its body must not be empty;
3. Randomly select a statement <i>s</i> from <i>m</i> ;
4. Extract <i>s</i> to a new method, <i>newMethod</i> . The new method's name is selected from a pool of names prepared in advance;
5. Insert the fault by removing a random statement <i>st</i> from <i>newMethod</i> ;
6. If the extracted method generates a compilation error, undo the extraction and return to Step 1.
Seeding a <i>relational operator replacement</i> fault in an Extract Method edit: Being <i>S</i> the set of classes from a given project version.
1. Randomly select a class <i>C</i> from <i>S</i> ;
2. Randomly select a method <i>m</i> from <i>C</i> ; its body must have at least a statement that uses a relational operator (<i>=</i> , <i>!=</i> , <i>></i> , <i><</i> , <i>>=</i> or <i><=</i>);
3. Randomly select a statement <i>s</i> from <i>m</i> that uses a relational operator;
4. Extract <i>s</i> to a new method, <i>newMethod</i> . The new method's name is selected from a pool of names prepared in advance;
5. If the extracted method generates a compilation error, undo the extraction and go back to step 1;
6. Insert the fault by replacing the operator from <i>s</i> with another one.
Seeding an <i>extra behavior</i> fault in an Extract Method edit: Being <i>S</i> the set of classes from a given project version.
1. Randomly select a class <i>C</i> from <i>S</i> ;
2. Randomly select a method <i>m</i> from <i>C</i> ; its body must have an access to a variable <i>v</i> ;
3. Randomly select a statement <i>s</i> from <i>m</i> that uses <i>v</i> ;
4. Extract <i>s</i> to a new method, <i>newMethod</i> . The new method's name is selected from a pool of names prepared in advance;
5. If the extracted method generates a compilation error, undo the extraction and go back to step 1;
6. In the original method, insert the fault by setting <i>v</i> to its default value before the call of the newly added method.

more than one version in each system presented test suites that miss a given fault.

Overall, 67% of the seeded faults were effectively detected by test suites; 33% of the erroneous refactorings pass all tests. The least effective test suite for those faults was XML-Security's – 53 out of 90 faults are missed; EasyAccept missed 23 out of 60. Better results were observed in JMock, with only 14 missing faults. Such results showed considerable lack of precision of the actual test suites, endorsing conclusions from previous research (e.g. Kim et al., 2012; Alves et al., 2013) reporting that, in practice, developers cannot fully trust their manual refactorings will be validated by tests. In fact, developing comprehensive test cases for a given project, especially for open source projects, – usually exhibiting a less controlled environment – can be challenging.

We can also group those results by refactoring and fault type (Table 3), then each combination *refactoring type X fault type* can be analyzed: similar rates of refactoring fault detection can be observed. For the Extract Method and Move Method edits, suites detected 69% and 64% of the faults. By fault type, less cases (61%) detected a *Missing Statement*, whereas 72% detected a *Relational Operator Replacement* and 67% detected a *Behavior change*.

In order to answer the first two research questions (RQ1 and RQ2), we performed a proportion hypothesis test, in which we compared whether the rates of each possible combination of grouping (refactoring type or fault) presented significant differences. Table 4 shows the proposed hypotheses; *C* refers to a suite's capacity of detecting refactoring faults, *EM* to an Extract Method edit, *MM* to a Move Method edit, *MS* to a Missing Statement fault, *ROR* to a Relational Operator Replacement fault, and *EM* to a Extra Behavior fault. We applied the Chi-Squared test, with a 95% confidence level. As a result, no significant difference was found in any of the tests. Thus, we can answer RQ1 and RQ2 by establishing

Table 2

Fault seeding on Move Method.

- Seeding a *statement deleting* fault in a move method edit:
Being S the set of classes from a given project version.
1. Randomly select a class C from S ;
 2. Randomly select a method m from C ; its body must not be empty;
 3. Randomly select a class C_2 ;
 4. Move m from C to C_2 ;
 5. Insert the fault by removing a random statement st from m in C_2 ;
 6. If a compilation error is found, undo the changes and go back to step 1.
- Seeding a *relational operator replacement* fault in a move method edit:
Being S the set of classes from a given project version.
1. Randomly select a class C from S ;
 2. Randomly select a method m from C ; its body must have at least a statement that uses a relational operator ($=$, $!$, $>$, $<$, $>=$ or $<=$);
 3. Randomly select a class C_2 ;
 4. Move m from C to C_2 ;
 5. If the moving generates a compilation error, undo the moving and go back to step 1;
 6. Randomly select a statement s from m that uses a relational operator;
 7. Insert the fault by replacing the operator from s with its negation.
- Seeding a *an extra behavior* fault in a move method edit:
Being S the set of classes from a given project version.
1. Randomly select a class C from S ;
 2. Randomly select a method m from C ; its body must have an access to a variable v ;
 3. Randomly select a class C_2 ;
 4. Move m from C to C_2 ;
 5. Insert the fault by removing a random statement st from m in C_2 ;
 6. Before the statement that accesses v , insert the fault by setting v to its default value.
 7. If a compilation error is found, undo the changes and go back to step 1.

Table 3

Contingency table for detection rates.

	MS (%)	ROR (%)	EB (%)
EM	58	76	73
MM	64	69	60
	61	72	67

Table 4

Null and alternative hypotheses regarding the proportion tests.

H0	H1	X-squared	p-value
$C_{EM} = C_{MM}$	$C_{EM} \neq C_{MM}$	0.4167	0.5186
$C_{MS} = C_{ROR} = C_{EB}$	$C_{MS} \neq C_{ROR} \neq C_{EB}$	2.5	0.2865
$C_{EM \wedge MS} = C_{EM \wedge ROR} = C_{EM \wedge EB}$	$C_{EM \wedge MS} \neq C_{EM \wedge ROR} \neq C_{EM \wedge EB}$	3.9401	0.1395
$C_{MM \wedge MS} = C_{MM \wedge ROR} = C_{MM \wedge EB}$	$C_{MM \wedge MS} \neq C_{MM \wedge ROR} \neq C_{MM \wedge EB}$	0.7759	0.6785
$C_{EM \wedge MS} = C_{MM \wedge MS}$	$C_{EM \wedge MS} \neq C_{MM \wedge MS}$	0.187	0.6654
$C_{EM \wedge ROR} = C_{MM \wedge ROR}$	$C_{EM \wedge ROR} \neq C_{MM \wedge ROR}$	0.2215	0.6379
$C_{EM \wedge EB} = C_{MM \wedge EB}$	$C_{EM \wedge EB} \neq C_{MM \wedge EB}$	1.25	0.2636

that, for those systems, the suites' capacity of revealing refactoring faults remained stable, not depending on type of refactoring nor type of fault.

Similarly, to verify whether the results are skewed by any of the subject programs, we calculated the rates of fault detection by program and performed an analysis similar to the one described in Table 4 (same null and alternative hypotheses for each program). After applying a Chi-Squared test, with a 95% confidence level, we verified that none of the three subjects (XML-Security, EasyAccept and JMock) presented any significant difference on the suites' capacity of revealing refactoring faults (X-squared values from 0.187 to 3.9401, with $df = 1$, and p -values from 0.1395 to 0.6785). This capacity remained stable for each of the program's suites in general, and when grouped by type of refactoring or type of fault.

General

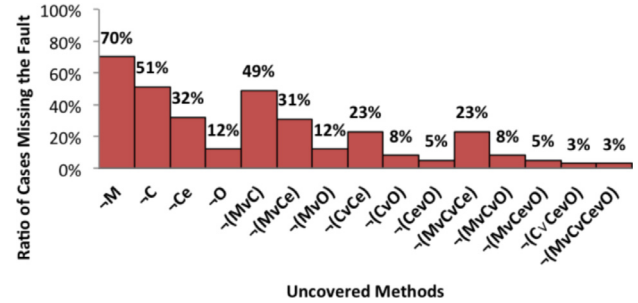


Fig. 7. Cases that miss faults when there is no first-level coverage of some indicated group of methods.

4.2. First-level coverage and undetected faults

In order to discuss RQ3, we related first-level coverage of test cases in commonly impacted methods (M , C , Ce and O) with a suite's results in detecting the seeded fault. First, we can relate the cases that *missed* the fault in terms of their first-level coverage of impacted elements (M , C , Ce and O). Fig. 7 shows, for all combinations of version and fault in general, the ratio of cases by which the faults were missed, by test suite, in terms of the method category not called directly by the suite's test cases. For instance, *the refactored method M is not called in 70% of cases that miss the fault*. The negation $\neg$ indicates the category of methods not covered. $\neg(M \vee C)$ represents the test suite does not cover either M or C .

The same results can be grouped into refactoring type (Extract Method or Move Method) or fault (Missing Statement, Relational Operator Replacement or Extra Behavior), as showed in Fig. 8). Our results show that the lack of calls to M seems to be very relevant – 70% of the unrevealed faults present this property – 71% for Extracted Method, 69% for Move Method, 54% for Missing Statement, 60% for Relational Operator Replacement, and 97% for Extra Behavior. Similarly, for 51% (EM: 55%, MM: 48%, MS: 40%, ROR: 40%, EM: 73%) of the undetected faults, the test suites do not call callers of M . The changed method and its callers were not called in 49% of the test cases (EM: 55%, MM: 44%, MS: 37%, ROR: 36%, EM: 73%) of the cases.

These data indicate, in this study, the chances of a suite missing refactoring faults are higher, if no test case directly calls the most impacted elements (M and C), regardless of the type of refactoring applied, or the type of seeded fault. In addition, techniques that generate test cases focusing on the impacted elements should be prioritized in refactoring scenarios; although this is visible to Extract Method and Move Method, other similar refactorings should benefit as well – Extract Class and Pull Up Method are representative examples.

Although less important, other combinations are worth discussing and should be considered by the tester. For instance, 23% of the test cases that miss the fault (EM: 19%, MM: 27%, MS: 17%, ROR: 8%, EB: 43%) cover neither C nor Ce . Thus, after assuring that both the changed method and its callers are well tested, if the tester has still time and resources available, it is worthy to head efforts in testing the method's callees in order to increase chances of detecting faults.

4.3. First-level coverage and detected faults

Also for all systems, we analyzed the cases in which the seeded fault was *detected* and relate them to first-level coverage of commonly impacted method (Fig. 9). Fig. 10 presents the same analysis by grouping the results by refactoring type and fault.

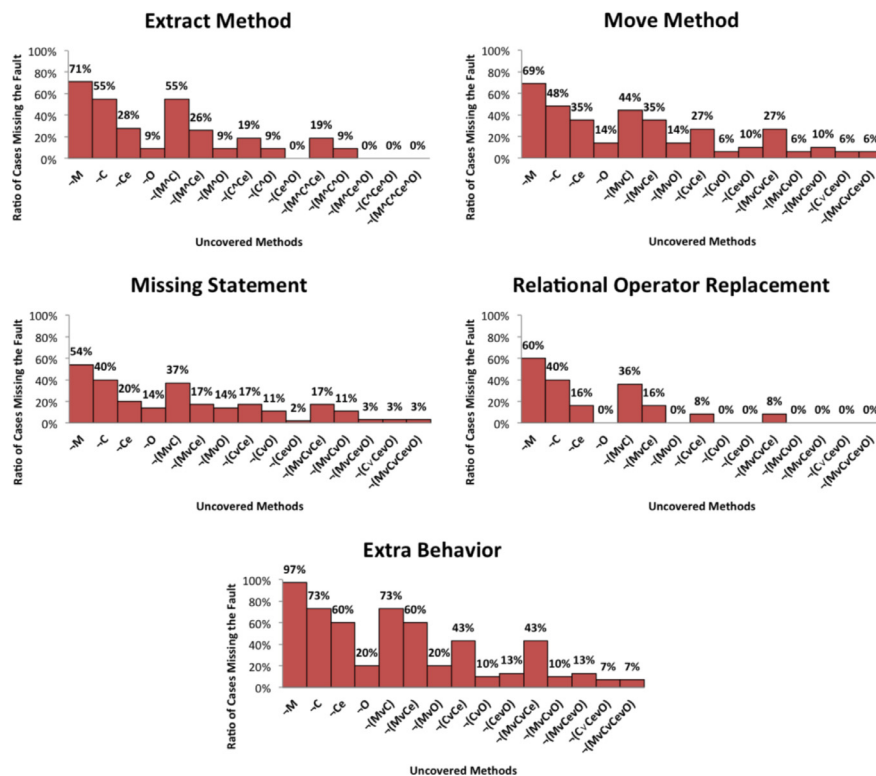


Fig. 8. Cases that miss faults, grouped by refactoring type or fault.

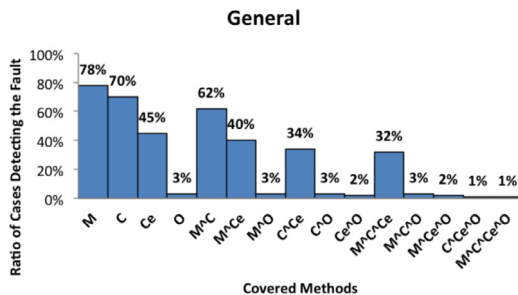


Fig. 9. Cases that detect faults when there first-level coverage of some indicated group of methods.

Likewise, our results evidence the importance of having tests that directly exercise the modified method. In 78% of the cases that identified a fault, there is at least one test case that directly calls the refactored method. Similar results are observed when analyzing the collected results separated by refactoring type (Extracted Method - 77%, Move Method - 78%) and seeded fault (Missing Statement - 93%, Relational Operational Replacement - 92%, Extra Behavior - 48%). Considering callers, this rate is also high - 70% (EM: 70%, MM: 71%, MS: 78%, ROR: 90%, EB: 42%), and the combination of M and C presents a rate of 62%. When analyzing the results individually, by type of refactoring and fault, the rates are high as well (EM: 42%, MM: 61%, MS: 74%, ROR: 85%, EB: 25%).

We can unify the views of undetected and detected cases by using the histogram charts in Fig. 11. Each chart relates first-level coverage of impacted methods to detection of faults (1, in the horizontal axis, means a fault is detected, and 0 undetected). The density curves clearly tend to be higher in 1 (fault detected) when test cases directly call M , C or $M \wedge C$. On the other hand, the density curves tend to be higher in 0 (undetected fault) for the other groups of methods and combinations. Moreover, the coefficients of

correlation for fault detection and M , C , and $M \wedge C$ (0.46; 0.32; and 0.36) are at least 2.5 times higher than all the other elements and their combinations. These results corroborate our previous conclusion that these two groups of impacted methods are the most relevant for test cases to call in case of refactoring edits.

Our results show that test cases that directly call methods with similar signature (O) have very little impact on testing/detecting refactoring faults. However, our fault seeding process emulated those scenarios in only 24 of the 270 cases (9%). As those situations are less common to happen, it is hard to emulate them in a non-manual seeding process - we randomly selected code elements such method, statements and method name to be part of the refactoring edits. Then, considering only those cases, tests that call those methods detected the fault only in four cases.

A few examples are worth mentioning (see details at our website.⁶) For instance, in the case where the first Extract Method MS fault was seeded to the third version of the XML-Security, we observed that there were several test cases that call both M and C . Similarly, in the fifth Extract Method MS fault seeded to JMock's code, we have a high number of test cases that exercise one of those groups (76 test cases covered C). Nevertheless, both scenarios missed the seeded Missing Statement fault. Those cases substantiate the claim that calling M or C is not a sufficient condition for detecting the refactoring fault, as can be seen in previous research results (e.g. Frankl and Weiss, 1991; Inozemtseva and Holmes, 2014). However, our results indicate that direct calls to those groups of methods are a good practice for maximizing detection of manually-introduced errors in refactoring.

In the aforementioned examples, we observe that although covering M and/or C , the tests fail to cover some branches of the refactored method, probably due to low testing data variability. That can mislead a judgment based on pure first-level coverage. In Section 5

⁶ <https://sites.google.com/site/coverageexperiment/>

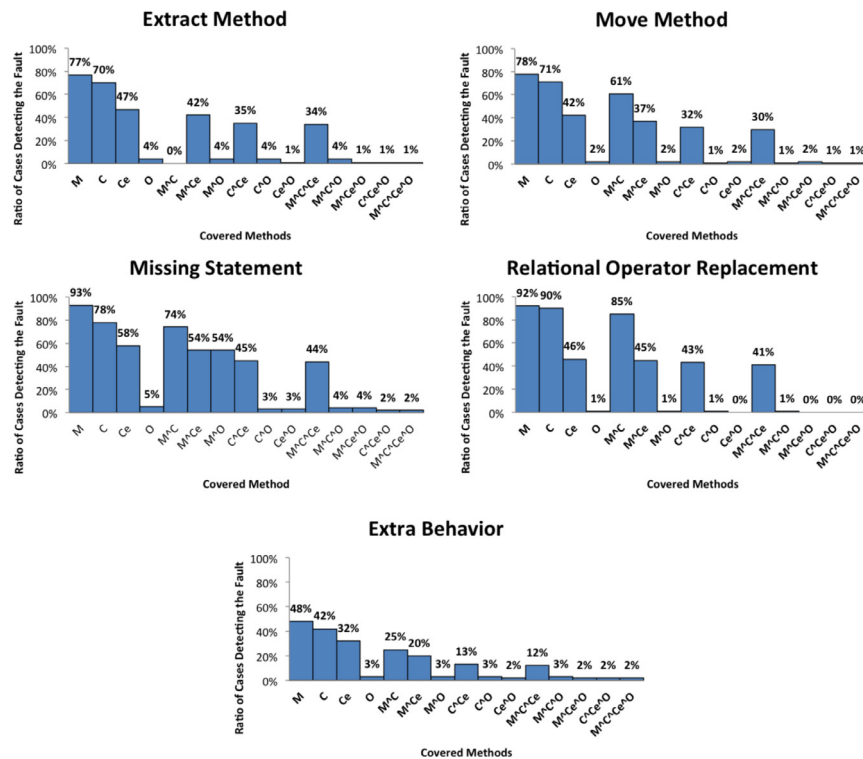


Fig. 10. Cases that detect faults, grouped by refactoring type or fault.

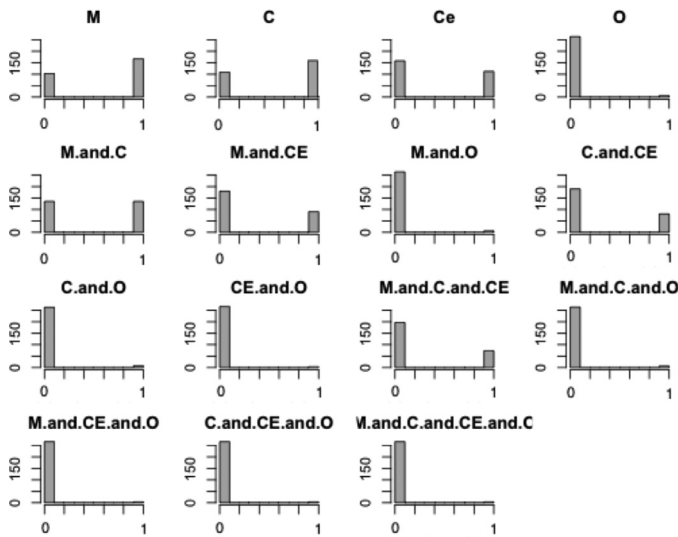


Fig. 11. Histogram charts relating fault detection and first-level coverage of impacted methods.

we investigate whether the combination of method and branch coverage increases the chance of detecting the fault.

4.4. A Statistical model for predicting detection of refactoring faults

The results build the basis for discussing two research question: RQ3 and RQ4. After observing the strong relationship between the first-level coverage of M , C , and/or $M \wedge C$ and refactoring fault detection, we propose a regression model based on our study data. This model summarizes direct call data and refactoring fault detection; the linear model provides a likely prediction for detection rate (response variable), based on the first-level coverage of M , C ,

Table 5

Regression statistics.

	Coefficient	p-value
Intercept	−0.8109	0.01942
M	3.1510	0.00059
C	2.6248	0.00220
$M: C$	−3.6752	0.00510

Ce and O (predictor variable). We use this model as the statistical foundation for discussing RQ3 and RQ4.

4.4.1. Effectiveness of first-level coverage in predicting faults

To build our prediction model we applied *Logistic Regression* (Hosmer et al., 2000; Field et al., 2012) by using a random sample of 50% of our data set results; we started at 10%, as usual in such analysis, and increased the sample until the model got considerably stable – when its R -squared value gets to its maximum value. This sample was increased until R -squared values were no greater than the greatest value previously found. For that, we used the R environment for statistical computing.⁷ The Logistic Regression is a statistical method used to predict a binary response from at least one categorical predictor. In our case, an observation is a test suite, the predictors (independent variables) refer to the coverage of certain impacted element (1 if the group of methods is called, 0 otherwise).

Table 5 shows the coefficients of each predictor from our model and their p -values, while Eq. (1) presents the proposed predictor model. Our model aims to infer whether a suite is likely to detect a refactoring fault. The closer to 1 is D , better are the chances of a refactoring fault being detected. Only M , C , and $M \wedge C$ proved to be essential; the other elements and combinations did not improve

⁷ <http://www.r-project.org/>

Table 6
Model confusion matrix.

		Reference	
		0	1
Prediction	0	30	10
	1	14	81

the model significantly (we used a bottom-up approach (Hosmer et al., 2000)), thus being removed without loss of precision. In the p-value column from Table 5, we can see that either M and C alone, and the interaction between the two ($M:C$), are significant, thus they all have effect over the detection result ($AIC = 147$).

$$D = 1/(1 + e^{-(0.8109 + 3.1510M + 2.6248C - 3.6758M \wedge C)}) \quad (1)$$

To verify the fit of our model we tested it using the remaining 50% of the data (no overlapping with the training set). With 95% of confidence, the model correctly predicted the detection of refactoring faults with accuracy of 0.82, within confidence interval [0.74; 0.88]. Table 6 presents the confusion matrix when we used our model against the reference data regards detection. Thus, we can say that our model showed to be effective by correctly predicting the detection of refactoring faults with high accuracy rates. Precision of this model can still be improved with further information such as different coverage analysis or other metrics. Next section presents an extension to this study in order to motivate further research in this direction.

We observed that in fact test coverage of commonly impacted locations is in general a good measure to predict a suite's capacity on detecting refactoring faults (RQ3). In addition, the lack of testing coverage on those elements can be consider a weakness, and be the starting point to guide developers to augment and improve a suite.

Similarly, our study identified that two classes of impacted elements (the method under refactoring and its callers, M and C) must be the first priority when testing an extracted method edit. Test cases that cover those elements are more likely to reveal refactoring faults (RQ4).

5. Variation: branch coverage analysis

As mentioned before, not all test suites that include test cases that cover M and/or C detect the fault. For instance, for one XML-Security faulty version, although its suite has 36 test cases that call M and C (37% of the suite), they do not detect the fault. In order to provide a more thorough analysis, and discuss RQ5, we extended the first study by selecting the subset of faults for which there was at least one test case directly calling the changed method and/or its callers ($M > 0$ and/or $C > 0$). In this scenario, we took a white-box approach to investigate which branches of the *original method* were covered by those test cases. For simplifying the analysis, we categorized two levels of branch coverage: [0; 25%] as low, and [75%; 100%] as high. By choosing these two levels we wish to observe separately two different views of the results, that is, when high coverage is related to detection and low coverage is related to non-detection. The ranges expresses the lack of precision we expected, that is, both 100% and 0% are unlikely to happen, but values near them are more realistic.

For instance, in the fifth Extract Method MS seeded fault to EasyAccept's code, the single test case that exercised the changed method (`Script.allErrorMessages()`) covers 100% of its branches (high branch coverage for M), while its callers, that are exercised by four test cases, cover 0% of `Script.allErrorMessages()` (low branch coverage for C). Table 7a summarizes the results for the selected cases.

Table 7
Branch coverage analysis.

(a) Analysis regarding fault detection and branch coverage levels.				
M, C or $M \wedge C$ + (Branch coverage $\geq 75\%$)	M	C	$M \wedge C$	
Ratio of detected faults	91%	88%	89%	
M, C or $M \wedge C$ + (Branch coverage $\leq 25\%$)	M	C	$M \wedge C$	
Ratio of detected faults	66%	62%	50%	
(b) Comparison between first-level coverage only and that combined with high branch coverage.				
Detection rate		Ratio of detected faults + High branch coverage	X-squared	p-value
	First level coverage			
$M \wedge C$	62%	89%	17.5237	2.837e-05
$M \vee C$	87%	94%	3.8517	0.04969

The combination of $M > 0$ and high branch coverage of the refactored method (greater than 75%) led the test suite to detect the fault in 91% of the cases. Similarly, 88% of the positive cases had $C > 0$, exercising more than 75% of the branches. Moreover, 89% of the test cases detected the fault with first level coverage of both M and C and a high branch coverage. On the other hand, in only 9%, 12% and 11% of the cases a high branch coverage was not associated with a fault detection, respectively.

When the branch coverage is low ($\leq 25\%$), this rate drops to approximately 66% (for C and C_{CM} it dropped to 62% and 50% respectively). So, for those three projects, a high branch coverage of the refactored method seems to increase the chances of detecting refactoring faults, when there is direct coverage of M and/or C .

After observing that by combining a first-level coverage of M, C or $M \wedge C$ with high branch coverage we increased the chances of detecting refactoring faults, we performed a second analysis, investigating the improvement of this combination when compared with first-level coverage of M, C or $M \wedge C$. Table 7b summarizes this analysis. First, we observed the proportion of test cases that detected faults when both M and C were called, not considering branch coverage (62%), and the rate of cases when they call those methods and a high branch coverage was found (89%). The combination ended up improving the fault detection by 27%. Similarly, there were cases that call M or C combined with a high branch coverage and there was fault detection (94%). When comparing these cases with other cases calling M or C , but not considering branch coverage, the rate of fault detection was 7% higher (87%). In order to discuss an answer to RQ5, we performed a Chi-squared proportion test comparing the two situations (only coverage and coverage plus high branch coverage) for the selected test cases. These tests have shown that, with 95% of confidence level, there are statistical differences between the proportions. Thus, we can state that the combination of coverage of impacted elements and high rates of branch coverage would in fact perform differently than coverage of impacted elements alone, so is the answer to RQ5. Moreover, the combined analysis ended up eliminating several false positives detected when using only the coverage of impacted elements. However, in some cases, although the mainly impacted elements are covered, the refactoring faults are not detected due to a low branch coverage.

Calling M and C directly might be misleading if taken in isolation. In some cases, even test cases calling M and C directly with a high branch coverage ($\geq 75\%$) miss faults. For instance, there was a case among the seeded ROR faults to the EasyAccept code that 11 test cases cover both M and C with a branch coverage of 100%, however those test cases do not reveal the seeded fault. Look-

```

1 class CanonicalizerBase {
2     byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet, String
        inclusiveNamespaces) ... {
3         if (this.xpathNodeSet.size() == 0) {
4             return new byte[0]; } ... } }
5 class Canonicalizer20010315Excl extends CanonicalizerBase {
6     byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet, String
        inclusiveNamespaces) ... {
7         try { this.renderedPrefixesForElement = new HashMap();
8             return super.engineCanonicalizeXPathNodeSet(xpathNodeSet);
9         } finally { ... } }
10    byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet) ... {
11        return this.engineCanonicalizeXPathNodeSet(xpathNodeSet, "");
12    } }
13 class Canonicalizer20010315ExclOmitComments{ ... }

```

(a) Original version.

```

1 class CanonicalizerBase {
2     byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet, String
        inclusiveNamespaces) ... {
3         if (this.xpathNodeSet.size() == 0) {
4             return new byte[0]; } ... } }
5 class Canonicalizer20010315Excl extends CanonicalizerBase{
6     byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet) ... {
7         return this.engineCanonicalizeXPathNodeSet(xpathNodeSet, "");
8     } }
9 class Canonicalizer20010315ExclOmitComments{
10    byte[] engineCanonicalizeXPathNodeSet(Set xpathNodeSet, String
        inclusiveNamespaces) ... {
11        try { this.renderedPrefixesForElement = new HashMap();
12            return super.engineCanonicalizeXPathNodeSet(xpathNodeSet);
13        } finally { ... } } }
14 }

```

(b) Code with a binding problem due to a Move Method.

Fig. 12. Binding problem example.

ing closely, these 11 test cases, although covering all branches, presented *low variability of test data*, which was the reason this fault could not be detected. Our results show evidence that calling *M* and/or *C* with high branch coverage increases the chance of revealing refactoring faults. However, the result is not free of false positives.

6. Study on binding-related faults

Reference binding problems are one of the most common faults developers face when refactoring. Since refactoring edits usually deal with the moving, renaming and replacement of source code elements, a developer may have problems with method and variable references and their overriding/overloading constraints. Recent studies emphasize how easy it is to include a binding problem when refactoring, even when using well-known automatic refactoring tools (Soares et al., 2011; Alves et al., 2014b).

Fig. 12 exemplifies a hypothetical fault on a Move Method applied to XML-Security. Method `Canonicalizer20010315Excl.engineCanonicalizeXPathNodeSet(Set, String)` is moved to `Canonicalizer20010315ExclOmitComments`. However, in the resulting program, the call within `Canonicalizer20010315Excl.engineCanonicalizeXPathNodeSet` refers to the method from `CanonicalizerBase`, rather than the original class. The lines pointed out refer to the location of the reference (binding) problem.

In order to investigate whether our previous conclusions hold for binding-related faults, we ran an additional exploratory study, only in the context of XML-Security, as it presents class hierarchies in several packages to better distribute behavior and avoid coupling. For this study, we manually created four faulty versions of a chosen version of the XML-Security code, by applying a single problematic Move Method to each version. Each Move Method edit ended up changing the program's behavior, by breaking/adding overriding/overloading of original methods. All faulty programs were compilable. Fig. 12 exemplifies a seeded fault used in our study.

We used the test suite for the system for collecting information about first-level coverage and ran it to measure the fault detection rate.

As a result, *all four faults were detected by the test suite*. For each faulty version, we then analyzed the first-level coverage of the mostly impacted elements (*M*, *C*, *Ce* and *O*). For all four cases, there were test cases directly calling both *M* and *C*. However, in one case, although having 18 test cases that called *C*, none of those detected the fault.

Furthermore, in all cases, there was at least one test that covers *O* and detected the faults. This fact suggests those methods may be more relevant for binding-related faults than general faults. They seem to be helpful for debugging binding-related faults as well, as they focus on methods with similar signatures, which is tailored to detect breaking/adding of override/overloading contracts. For instance, considering the fault in Fig. 12, one of the test cases that revealed this fault exercised the method `engineCanonicalizeXPathNodeSet(Set)`. By acknowledging this test case, a developer can infer that the edit may have added or broken an overloading/overriding contract, which in fact happened in this example.

As a conclusion, regarding RQ6, the results of this small study show evidences that our previous conclusions might be still valid when detecting binding-related faults. *M* and *C*, if directly called, increase the chances of detecting those faults. However, in this particular context, other impacted methods (e.g., *O*) play an important detection role.

7. Threats to validity

In terms of *internal validity*, the accuracy of Eclemma and the coded R functions in calculating coverage and statistics might directly affect our study results. However, both tools have been widely used in practice, and applied by researchers, which attests that they rely on their results. Moreover, we manually validated Eclemma's and our regression functions' results for a limited sample. Likewise, we measured only first-level coverage for the first study, and branch coverage as an extension, which can be an issue to *construct validity*. The construct *effectiveness of test coverage* might as well be represented by other coverage metrics (e.g. decision coverage), and testing data, which could influence results. Moreover, coverage of indirect calls to impacted elements can also provide additional information on effectiveness of the test suite. We plan to investigate the impact of those factors as future work.

Regarding *external validity*, our evaluation results certainly do not generalize beyond the three studied projects. However, due to the projects' variability in size and test suites, we believe they are good representatives of the universe of open source Java programs. In the less controlled environment of an open source project, it is usually harder to build and maintain an effective test suite. Industrial projects, on the other hand, with tighter QA practices, might present different results. In addition, we focused on the Extract and Move Method refactorings. The effect of coverage to other refactoring types depend on the methods impacted by the change. Nevertheless, several of the most applied refactoring edits – such

as Pull Up Method or Encapsulate Field (Fowler et al., 1999) – involve creating new methods, and adding new calls to them, thus results might be comparable.

One can argue the applied seeded faults do not represent typical mistakes in manual refactoring. However, the three types of faults seeded can be found in actual systems, such as presented in the study by Bavota et al. (2012). This work detects several refactorings that have gone wrong somehow, giving rise to actual bugs in future versions. We went through all such bugs, and identified several examples for each of the types of faults we seeded in our studies. Again, we lack in generality, by exploring only three types of faults. However, we believe seeded faults are representative of actual ones.

Our study manipulates 270 seeded faults. This limitation reduces the generalization power of our study. Due to the kinds of subtle faults we are studying, which maintain the programs *compatible*, fault seeding is more costly. The process discards any output program with compilation errors, which drastically reduces the possible useful mutant versions. Moreover, due to the lack of tools to automatically insert the refactoring faults we are interested in (for instance, the lack of mutation operators related to refactoring), we need to follow a systematic procedure to manually find a fault seeding opportunity and insert it when performing a refactoring, that do not preserve behavior. However, besides generality, which is often difficult or even impossible to achieve in most studies, particularly when human errors are targeted, as in our context, our main contribution is to provide evidence on the use of common tester views and intuitions (regarding first-level coverage) and leave experimental design and data that can be applied to extend and/or reproduce the study.

It is important to remark that related studies present similar configuration problems. For instance, Frankl and Iakounenko's experiment (Frankl and Iakounenko, 1998) uses a set of 33 versions of a program, each one with a single seeded fault. Gligoric et al. (2013), although working with simpler mutant operators, create less than 60 mutant versions for several subjects. Although 426 mutant were created for Cai and Lyu's experiment (Cai and Lyu, 2005), a single program is used, with a suite generated for experimental purposes. Our study applies a more practical analysis by using real world artifacts: three open source programs with genuine test suites. We believe that this configuration enables a satisfactory relationship to real-world scenarios.

8. Related work

8.1. Testing coverage and suite effectiveness

Long-established texts on software testing (e.g. Perry, 2006) recommend the use of coverage, in general, to gain confidence that a test suite is effective on detecting faults. Extensive research on this topic, however, presents divergent conclusions. Frankl and Iakounenko (1998) report the chance of detecting a fault increases sharply with very high coverage rates. Wong et al. (1994) find a suite effectiveness is highly correlated with block coverage, Cai and Lyu (2005) observe a moderate correlation between coverage and fault detection, depending on the tests nature. Furthermore, Gligoric et al. (2013) state that fault detection effectiveness is moderately correlated with coverage, in general, and project size.

On the other hand, Frankl and Weiss (1991) find a weak correlation between suite coverage and its error-exposing ability. Later work Frankl et al. (1997) investigates the relationship between coverage and effectiveness for fixed-sized randomly generated test suites, and an improvement in effectiveness with higher coverage is observed (although there is no nonlinear relation). Briand and Pfahl (1999) do not find any causal dependency between test coverage and defect coverage with respect to four coverage

criteria (block, c-use, decision, p-use), independently of suite size. Inozemtseva and Holmes (2014) find there is a low to moderate correlation between coverage and suite effectiveness, and that stronger forms of coverage do not provide better data to measure effectiveness of a suite. Our work differs from all of the above by focusing its investigation on the role of *first-level coverage* to evaluate a suite, specific for refactoring. In Alves (2012a, 2012b) we investigate how coverage data work for anticipating refactoring faults detection. These studies have shown that coverage by itself does not provide enough information for relating refactoring faults and test cases. Thus, rather than considering coverage data as it is, our exploratory study observe direct calls to specific groups of possibly impacted routines.

Gargantini et al. (2012) propose a framework to evaluate the robustness of a test suite with respect to semantic preserving transformations. For that, they define new coverage metrics that consider the fragility of coverage. Our study applies a different approach by using direct calls to commonly impacted methods to predict a suite's capability of detecting a refactoring fault. Furthermore, our study's results are based on artifacts from real projects, and may be used as reference pinpointing code locations that might be emphasized for testing refactorings.

8.2. Refactoring and testing

Refactoring edits are very common during software development. Xing and Stroulia (2005) find that 70% of structural changes developing the Eclipse IDE are refactorings. Those edits are mostly manually performed Murphy et al. (2006). Negara et al.'s study Negara et al. (2013) finds that experts still prefer manual refactorings over automated. However, refactoring can be an error-prone activity. Kim et al. (2011) find there is correlation between the timing and location of refactorings and bug fixes. Weißgerber and Diehl (2006) find that, as more refactorings are applied, the number of bug reports increases. 77% of the participants of Kim et al.'s field study Kim et al. (2012) recognize that refactoring activities raise the risk of bug introduction. Daniel et al. (2007) and Soares et al. (2013) find several bugs in well-known automatic refactoring tools.

Several strategies and tools have been proposed for validating refactoring Soares et al. (2010); Cornélio et al. (2010); Ge and Murphy-Hill (2014); Alves et al. (2014b), however regression tests still play an important role. Kim et al. (2012) find that, when a regression test suite is inadequate, it could prevent developers from initiating refactoring, and regression testing suites, in practice, are often inadequate for covering refactored locations. Also, change impact and its relation with testing have been under investigation recently. Ren et al. (2004) proposes a strategy to decompose the differences of two program versions into atomic changes, and use analysis of test cases' call graphs with rules for identifying a subset of impacted tests. Zhang et al. (2011) extend Ren et al.'s work by improving their impact analysis, and introducing a spectrum analysis that helps inspect and rank the changes according to their chances of localizing faults. Alves et al. (2013) propose refactoring fault rules that relate the most commonly affected locations by a refactoring and use their coverage to prioritize test cases. To the best of our knowledge, our work is the first to combine impacted methods and first-level coverage to evaluate a suite effectiveness and give warnings regarding suite weaknesses with respect to refactoring fault detection.

9. Concluding remarks

This article reports studies on the relationship between program methods impacted by the Extract Method or Move Method refactoring and first-level test coverage of impacted elements.

Three real open-source Java systems were subject to seeded refactoring faults. Using the actual test suites employed by the selected projects, we measured the coverage of several groups of methods impacted by a refactoring edit, relating these data to the status of the test case (whether it detects or not the seeded fault). Our exploratory studies provide evidences that, by directly exercising two different groups of impacted methods – the modified method and its callers –, chances are higher that the refactoring faults related to both Extract Method and Move Method edits get detected. Also, if the test suite additionally achieves a high branch coverage of the extracted (or moved) method, chances of detecting those faults increase. On the other hand, the lack of direct calls to those groups of methods may strongly decrease a suite's capability for detecting refactoring faults, and for practical reasons, we focus our investigation on first-level coverage for assessing the suite's capability of detecting refactoring faults. We derived a statistical model that aims to help developers to predict whether a test suite is likely to detect the refactoring fault, based on the type of method called by the test suite.

A few guidelines to developers, obtained from the results of the reported studies, seem appropriate. When a developer is performing an Extract Method or Move Method edit, and she mistrusts the test suite as a safety net for avoiding behavioral changes, it seems wise to perform a previous analysis on the test suite. In this analysis, she might identify whether the two main groups of impacted elements – the changed method and its callers – are directly exercised. Then, by applying the proposed statistical model, she can obtain an appropriate approximation for the effectiveness of the test suite. This information can be used to help her decide whether it is safe to perform the refactoring edit, or whether there is room for a test suite improvement. Moreover, by knowing the locations not efficiently tested (e.g. low branch coverage), she may direct efforts to developing tests that improve this deficiency.

As future work, we plan to: (i) investigate the generality of our conclusions by expanding the study using different refactoring edits and types of faults; (ii) use data from new applications, new coverage metrics, and/or heuristics (e.g., number of test cases that cover impacted methods or rate of statements covered) to refine our prediction model; (iii) investigate whether system characteristics (e.g. coupling and method cyclomatic complexity) are factors that can be used to prioritize test cases, aiming to speed up the detection of refactoring faults; (iv) investigate whether the proposed model could also be used for evaluating and improving a test suite for validating non-refactoring changes; and (v) come up with variations of the prediction model, for instance, by training the model on numeric counts for M or C , resulting in more data about methods that might need more testing if a suite is augmented.

Acknowledgments

This work was partially supported by the National Institute of Science and Technology for Software Engineering, funded by CNPq/Brasil, Grant 573964/2008-4.

References

- Alves, E., 2012a. Investigating Test Case Prioritization Techniques for Refactoring Activities Validation: Evaluating Suite Characteristics. Technical Report SPLab-2012-002. Software Practices Laboratory, UFCG. <http://splab.computacao.ufcg.edu.br/technical-reports/>
- Alves, E., 2012b. Investigating Test Case Prioritization Techniques for Refactoring Activities Validation: Evaluating the Behavior. Technical Report SPLab-2012-001. Software Practices Laboratory, UFCG. <http://splab.computacao.ufcg.edu.br/technical-reports/>
- Alves, E.L., Machado, P.D., Massoni, T., 2014. Test coverage and impact analysis for detecting refactoring faults: A study on the extract method refactoring. Proceedings of the 30th ACM/SIGAPP Symposium on Applied Computing - SAC 2014 (To appear).
- Alves, E.L., Machado, P.D., Massoni, T., Santos, S.T., 2013. A refactoring-based approach for test case selection and prioritization. Proceedings of the 2013 8th International Workshop on Automation of Software Test (AST). IEEE, pp. 93–99.
- Alves, E.L., Song, M., Kim, M., 2014. Reddistiller: A refactoring aware code review tool for inspecting manual refactoring edits. Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, Research Demonstration Track (To appear). ACM.
- Andrews, J.H., Briand, L.C., Labiche, Y., 2005. Is mutation an appropriate tool for testing experiments? [software testing]. Proceedings of the 27th International Conference on Software Engineering, 2005. ICSE 2005. IEEE, pp. 402–411.
- Andrews, J.H., Zhang, Y., 2003. General test result checking with log file analysis. *Softw. Eng., IEEE Trans.* 29 (7), 634–648.
- Bavota, G., De Carluccio, B., De Lucia, A., Penta, M.D., Oliveto, R., Strollo, O., 2012. When does a refactoring induce bugs? an empirical study. Proceedings of the IEEE 12th International Working Conference on Source Code Analysis and Manipulation (SCAM), 2012. IEEE, pp. 104–113.
- Briand, L., Pfahl, D., 1999. Using simulation for assessing the real impact of test coverage on defect coverage. Proceedings of the 10th International Symposium on Software Reliability Engineering, 1999. IEEE, pp. 148–157.
- Briand, L.C., Labiche, Y., Wang, Y., 2004. Using simulation to empirically investigate test coverage criteria based on statechart. Proceedings of the 26th International Conference on Software Engineering. IEEE Computer Society, pp. 86–95.
- Cai, X., Lyu, M.R., 2005. The effect of code coverage on fault detection under different testing profiles. *ACM SIGSOFT Software Engineering Notes*, vol. 30. ACM, pp. 1–7.
- Cornélio, M., Cavalcanti, A., Sampaio, A., 2010. Sound refactorings. *Science of Computer Programming* 75 (3), 106–133.
- Daniel, B., Dig, D., Garcia, K., Marinov, D., 2007. Automated testing of refactoring engines. Proceedings of the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering. ACM, pp. 185–194.
- Dig, D., Johnson, R., 2005. The role of refactorings in api evolution. Proceedings of the 21st IEEE International Conference on Software Maintenance, 2005, ICSM'05. IEEE, pp. 389–398.
- Field, A., Miles, J., Field, Z., 2012. Discovering Statistics Using r.
- Fowler, M., Beck, K., Brant, J., Opdyke, W., Roberts, D., 1999. Refactoring: Improving the Design of Existing Programs.
- Frankl, P.G., Iakoumenko, O., 1998. Further empirical studies of test effectiveness. *ACM SIGSOFT Software Engineering Notes*, vol. 23. ACM, pp. 153–162.
- Frankl, P.G., Weiss, S.N., 1991. An experimental comparison of the effectiveness of the all-uses and all-edges adequacy criteria. Proceedings of the Symposium on Testing, Analysis, and Verification. ACM, pp. 154–164.
- Frankl, P.G., Weiss, S.N., Hu, C., 1997. All-uses vs mutation testing: an experimental comparison of effectiveness. *J. Syst. Softw.* 38 (3), 235–253.
- Gargantini, A., Guarnieri, M., Magri, E., 2012. Extending coverage criteria by evaluating their robustness to code structure changes. In: *Testing Software and Systems*. Springer, pp. 168–183.
- Ge, X., Murphy-Hill, E., 2014. Manual refactoring changes with automated refactoring validation. Proceedings of the 36th International Conference on Software Engineering. ACM, pp. 1095–1105.
- Gligoric, M., Groce, A., Zhang, C., Sharma, R., Alipour, M.A., Marinov, D., 2013. Comparing non-adequate test suites using coverage criteria. Proceedings of the 2013 International Symposium on Software Testing and Analysis. ACM, pp. 302–313.
- Hosmer, D.W., Lemeshow, S., Sturdivant, R.X., 2000. Introduction to the Logistic Regression Model. Wiley Online Library.
- Inozemtseva, L., Holmes, R., 2014. Coverage is not strongly correlated with test suite effectiveness. *ICSE*, pp. 435–445.
- Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing? Proceedings of the ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE 2014). ACM.
- Kim, M., Cai, D., Kim, S., 2011. An empirical investigation into the role of api-level refactorings during software evolution. Proceedings of the 33rd International Conference on Software Engineering. ACM, pp. 151–160.
- Kim, M., Zimmermann, T., Nagappan, N., 2012. A field study of refactoring challenges and benefits. Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering. ACM, p. 50.
- Kim, S.-W., Clark, J.A., McDermid, J.A., 2001. Investigating the effectiveness of object-oriented testing strategies using the mutation method. *Softw. Test., Verif. Reliab.* 11 (4), 207–225.
- Lee, Y.Y., Chen, N., Johnson, R.E., 2013. Drag-and-drop refactoring: intuitive and efficient program transformation. Proceedings of the 2013 International Conference on Software Engineering. IEEE Press, pp. 23–32.
- Memor, A., Banerjee, I., Nagarajan, A., 2003. What test oracle should i use for effective gui testing? Proceedings of the 18th IEEE International Conference on Automated Software Engineering, 2003.. IEEE, pp. 164–173.
- Mens, T., Tourwé, T., 2004. A survey of software refactoring. *Softw. Eng., IEEE Trans.* 30 (2), 126–139.
- Moser, R., Abrahamsson, P., Pedrycz, W., Sillitti, A., Succi, G., 2008. A case study on the impact of refactoring on quality and productivity in an agile team. In: *Balancing Agility and Formalism in Software Engineering*. Springer, pp. 252–266.
- Murphy, G.C., Kersten, M., Findlater, L., 2006. How are java software developers using the ellipse ide? *Softw., IEEE* 23 (4), 76–83.
- Murphy-Hill, E., Parnin, C., Black, A.P., 2012. How we refactor, and how we know it. *Softw. Eng., IEEE Trans.* 38 (1), 5–18.

- Negara, S., Chen, N., Vakilian, M., Johnson, R.E., Dig, D., 2013. A comparative study of manual and automated refactorings. In: ECOOP 2013–Object-Oriented Programming. Springer, pp. 552–576.
- Perry, W.E., 2006. Effective Methods for Software Testing: Includes Complete Guidelines, Checklists, and Templates. John Wiley & Sons.
- Ren, X., Shah, F., Tip, F., Ryder, B.G., Chesley, O., 2004. Chianti: a tool for change impact analysis of java programs. ACM Sigplan Notices, vol. 39. ACM, pp. 432–448.
- Ryder, B.G., Tip, F., 2001. Change impact analysis for object-oriented programs. Proceedings of the 2001 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering. ACM, pp. 46–53.
- Soares, G., Catao, B., Varjao, C., Aguiar, S., Gheyi, R., Massoni, T., 2011. Analyzing refactorings on software repositories. 25th Brazilian Symposium on Software Engineering (SBES), 2011. IEEE, pp. 164–173.
- Soares, G., Gheyi, R., Massoni, T., 2013. Automated behavioral testing of refactoring engines. Softw. Eng., IEEE Trans. 39 (2), 147–162.
- Soares, G., Gheyi, R., Serey, D., Massoni, T., 2010. Making program refactoring safer. Softw., IEEE 27 (4), 52–57.
- Weißgerber, P., Diehl, S., 2006. Are refactorings less error-prone than other changes? Proceedings of the 2006 International Workshop on Mining Software Repositories. ACM, pp. 112–118.
- Wong, W.E., Horgan, J.R., London, S., Mathur, A.P., 1994. Effect of test set size and block coverage on the fault detection effectiveness. Proceedings of the 5th International Symposium on Software Reliability Engineering, 1994. IEEE, pp. 230–238.
- Xing, Z., Stroulia, E., 2005. Umldiff: an algorithm for object-oriented design differencing. Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering. ACM, pp. 54–65.
- Zhang, L., Kim, M., Khurshid, S., 2011. Localizing failure-inducing program edits based on spectrum information. Proceedings of the 27th IEEE International Conference on Software Maintenance (ICSM), 2011. IEEE, pp. 23–32.

Everton L.G. Alves is a doctoral candidate in the Computing and Systems Department at Federal University of Campina Grande (UFCG), Brazil. He received a Master degree in Computer Science from the Federal University of Campina Grande, Brazil, in 2011 and a Bachelor degree in Computer Science in 2009, from the same university. His main interests include regression testing, software maintenance, model-driven development and testing, real-time systems and integration.

Tiago Massoni is a professor in the Computing and Systems Department at the Federal University of Campina Grande. His research interests include software design and evolution, and formal methods. In addition to his academic posts he also worked as a programmer at IBM in California. He holds a Doctoral degree in Computer Science from the Federal University of Pernambuco, and is a member of the ACM.

Patrícia Duarte de Lima Machado is a Professor in the Computing and Systems Department at Federal University of Campina Grande (UFCG), Brazil, since 1995. She received her PhD Degree in Computer Science from the University of Edinburgh, UK, in 2001, Master Degree in Computer Science from the Federal University of Pernambuco, Brazil, in 1994 and Bachelor Degree in Computer Science from the Federal University of Paraíba, Brazil, in 1992. Her research interests include software testing, formal methods, mobile computing, component based software development and model-driven development. Since 1998, she has produced a number of contributions in the area of software testing, including research projects, publications, tools, supervising, national/international cooperation and teaching.